

Current Controller Design Wizard

Calculation & Design Report (v3)

Author: Masoud Bakhshi

Date: 2026-02-16

Tool version: V1

Executive summary

This report documents the end-to-end design flow implemented in the “Current Controller Design Wizard” for dq-axis current control using a delay-aware bandwidth PI method and robustness verification via Monte Carlo parameter sweeps. It is written to be audit-ready: equations are shown with units, intermediate numeric steps are included, and each wizard output plot is explained with practical interpretation guidance for calibration, dyno, and vehicle deployment.

Scope note: The tuning plant used by the wizard is a per-axis RL model (dq decoupled). Real IPMSM effects (cross-coupling, back-EMF, saliency vs current, inverter non-idealities, filters) are addressed in the “Engineering notes & limitations” section, and should be validated on dyno/vehicle logs.

How to read this report

This section explains what the wizard does end-to-end, where numbers come from, and what is model-based vs validated.

End-to-end flow (inputs → computations → outputs)

Inputs: Machine RL parameters (R_s , L_d , L_q), sampling rate, delay budget (PWM/ZOH + compute + measurement), DC-link + PWM method/limits, and the user-selected target crossover / phase-margin intent.

Computations: Per-axis RL plant model → delay model (Padé/phase rule) → bandwidth PI gains (K_p , K_i) → discrete PI coefficients (Tustin incremental form) → stability metrics (PM/GM) and sensitivity peaks (M_s/M_t) → saturation/anti-windup time-domain checks → robustness Monte Carlo sweep.

Outputs: Proposed gains and discrete coefficients, Bode plots for plant/open-loop/sensitivity, margin metrics, Monte Carlo distributions, and time-domain saturation/anti-windup responses.

Single source of truth

This report is tied to the provided Python implementation (CurrentCtrlWizard.py) and the provided wizard screenshots. No additional parameters are assumed. If a parameter is required for a derivation but not present in code/config/UI, it is marked as TBD, and the exact missing field is stated.

Model-based approximations vs validated results

Model-based: RL plant magnitude/phase, delay modeling, analytical gain formulas, frequency-domain margins/peaks, and the robustness sweep (which depends on assumed uncertainty ranges). Validated (requires measurement): actual closed-loop bandwidth and ripple in logs, saturation frequency/shape under voltage limits, effective delay (measurement + compute + PWM), and inductance variation vs current/temperature. The wizard helps you predict these effects and design a safe starting point, but it does not replace dyno/vehicle validation.

Project setup (single source of truth)

The following parameters are extracted from the provided code defaults and wizard screenshots.

Machine / electrical plant parameters

Parameter	Symbol	Value	Units / Notes
Stator resistance	R_s	0.002000	Ω (per-phase, dq-axis equivalent in RL model)
d-axis inductance	L_d	0.000300000	H
q-axis inductance	L_q	0.000450000	H
PM flux linkage	ψ_f	0.000000	Wb (optional; not used in RL tuning model; affects back-EMF/decoupling in real system)

Inverter / PWM / limits/timing

Parameter	Symbol	Value	Units / Notes
DC link voltage	V_{dc}	700.0	V
PWM method	-	SVPWM	Used only for the voltage limit model
Max modulation index	m_{max}	0.95	—
Voltage limit (phase peak)	$V_{phase_peak, max}$	383.938	V (SVPWM: $V_{max} = m_{max} \cdot V_{dc} / \sqrt{3}$)
Sampling frequency	f_s	10000.0	Hz
Sampling time	T_s	0.0001000	s
PWM/ZOH delay	T_{pwm}	0.00005000	s
Computation delay	T_{comp}	0.00005000	s
Measurement delay	T_{meas}	0.00005000	s
Total delay (budget)	T_d	0.00015000	s ($T_d = T_{pwm} + T_{comp} + T_{meas}$)

Control objective & architecture

Objective (dq current loop)

Primary objective: track i_d^* and i_q^* with high bandwidth while maintaining robust stability under delay, parameter uncertainty, and voltage saturation. Secondary objectives: reject disturbances (e.g., back-EMF, coupling, inverter voltage errors) and avoid excessive noise amplification that would increase current ripple and EMC stress.

Architecture implemented in the wizard

The wizard tunes two independent PI controllers (d-axis and q-axis) around a per-axis RL plant model. The design explicitly accounts for an aggregated delay budget. The saturation/anti-windup check evaluates integrator windup risk and step-response behavior under a voltage limit computed from Vdc, PWM method, and modulation headroom.

Block diagram (conceptual)

$i^* \rightarrow [\Sigma] e = (i^* - i) \rightarrow \text{PI}(z) \rightarrow v_{\text{cmd}} \rightarrow \text{saturation } (V_{\text{max}}) \rightarrow v_{\text{applied}} \rightarrow \text{plant } G(s) \rightarrow i$
 $\hookrightarrow$ anti-windup logic (clamp / back-calc)

Plant model used for tuning

Per-axis RL model derivation (d- and q-axes)

For tuning, each axis is modeled as a first-order RL circuit (dq decoupled). Using SI units:

$$v_{d(t)} = R_s \cdot i_d(t) + L_d \cdot di_d(t)/dt \quad [V = \Omega \cdot A + H \cdot A/s]$$

$$v_{q(t)} = R_s \cdot i_q(t) + L_q \cdot di_q(t)/dt \quad [V = \Omega \cdot A + H \cdot A/s]$$

Taking Laplace transforms (zero initial conditions):

$$I_d(s)/V_d(s) = 1 / (L_d \cdot s + R_s) \quad [A/V]$$

$$I_q(s)/V_q(s) = 1 / (L_q \cdot s + R_s) \quad [A/V]$$

With the provided parameters: $\tau_d = L_d/R_s = 0.000300000/0.002000 = 1.500000\text{e-}01$ s, $\tau_q = L_q/R_s = 0.000450000/0.002000 = 2.250000\text{e-}01$ s.

Delay model used in stability calculations

The wizard models implementation delay as an aggregated time delay T_d . In frequency-domain reasoning, this contributes a phase lag of $-\omega \cdot T_d$ (radians). In some plots/metrics, a Padé approximation may be used

for transfer function calculations; the phase-margin rule below matches what the wizard uses to compute the delay-limited crossover.

The controller design method used

Delay-aware bandwidth PI (pole/zero cancellation + crossover placement)

The implemented tuning method is a classical bandwidth PI for first-order plants, constructed so that the PI zero approximately cancels the RL pole, leaving an integrator-like open-loop with a selected crossover frequency.

Continuous-time controller form

$$C(s) = K_p + K_i/s \quad [V/A]$$

Open loop with RL plant (no delay yet)

For one axis with inductance L and resistance R:

$$L(s) = C(s) \cdot G(s) = (K_p + K_i/s) \cdot 1/(L \cdot s + R)$$

$$L(s) = (K_p \cdot s + K_i) / (s \cdot (L \cdot s + R))$$

Pole-zero cancellation condition (why $K_i/K_p = R/L$)

Factor the numerator as $K_p \cdot (s + K_i/K_p)$. The plant pole is at $s = -R/L$. Choosing:

$$K_i/K_p = R/L \quad [1/s]$$

aligns the PI zero ($-K_i/K_p$) with the plant pole ($-R/L$). This is an approximation in real IPMSM systems (because L varies with current and cross-coupling exists), but it is a strong starting point and yields a predictable closed-loop shape.

Crossover placement (why $K_p = L \cdot \omega_c$ and $K_i = R \cdot \omega_c$)

With exact cancellation, $(L \cdot s + R) \approx L \cdot (s + R/L)$, and $(K_p \cdot s + K_i) = K_p \cdot (s + R/L)$. The open loop simplifies to:

$$L(s) \approx K_p / (L \cdot s)$$

Therefore $|L(j\omega)| \approx (K_p/L) \cdot (1/\omega)$. Setting the desired crossover $\omega = \omega_c$ at $|L(j\omega_c)| = 1$ gives:

$$K_p/L = \omega_c \Rightarrow K_p = L \cdot \omega_c$$

$$K_i = (R/L) \cdot K_p = (R/L) \cdot (L \cdot \omega_c) = R \cdot \omega_c$$

Worked numeric example (this project)

Selected crossover: $f_c = 500.0 \text{ Hz} \Rightarrow \omega_c = 2\pi \cdot f_c = 2\pi \cdot 500.0 = 3141.593 \text{ rad/s}$.

d-axis gains:

$$Kp_d = L_d \cdot \omega_c = 0.000300000 \cdot 3141.593 = 0.942478 \text{ V/A}$$

$$Ki_d = R_s \cdot \omega_c = 0.002000 \cdot 3141.593 = 6.283185 \text{ V/(A}\cdot\text{s)}$$

q-axis gains:

$$Kp_q = L_q \cdot \omega_c = 0.000450000 \cdot 3141.593 = 1.413717 \text{ V/A}$$

$$Ki_q = R_s \cdot \omega_c = 0.002000 \cdot 3141.593 = 6.283185 \text{ V/(A}\cdot\text{s)}$$

Delay-limited crossover calculation

Why it matters: even if the RL model is benign, real current loops are delay-dominated. Above a certain bandwidth, the extra phase lag from delay collapses the phase margin and the loop becomes oscillatory (seen as ripple and/or instability).

Phase-margin rule used by the wizard

With RL pole-zero cancellation, the open-loop behaves approximately like an integrator (-90° phase) plus delay ($-\omega \cdot T_d$). At crossover ω_c , the open-loop phase is approximately:

$$\angle L(j\omega_c) \approx -90^\circ - (\omega_c \cdot T_d) \cdot (180/\pi)$$

Therefore, the phase margin is:

$$PM \approx 180^\circ + \angle L(j\omega_c) \approx 90^\circ - (\omega_c \cdot T_d) \cdot (180/\pi)$$

Rearranging for a target PM_{target} gives the delay-limited crossover bound:

$$\omega_{c, \text{max}} \approx ((90^\circ - PM_{\text{target}}) \cdot (\pi/180)) / T_d = (\pi/2 - PM_{\text{target_rad}}) / T_d$$

Worked numeric example (this project)

Total delay: $T_d = 0.00015000$ s. Target PM: $60.0^\circ \Rightarrow PM_{\text{target_rad}} = 1.047198$ rad.

$$\omega_{c, \text{max}} = (\pi/2 - PM_{\text{target_rad}}) / T_d = (1.570796 - 1.047198) / 0.00015000 = 3490.659 \text{ rad/s}$$

$$f_{c, \text{max}} = \omega_{c, \text{max}} / (2\pi) = 3490.659 / (2\pi) = 555.6 \text{ Hz.}$$

The wizard uses $f_c = 500.0$ Hz, which is below the delay-limited maximum $f_{c, \text{max}} \approx 555.6$ Hz, preserving margin.

Discrete-time implementation

Tustin incremental PI coefficients

The wizard exports an incremental (velocity-form) PI using Tustin (bilinear) discretization, which is commonly preferred in ECUs because it is numerically stable and easy to saturate/anti-windup.

$$u[k] = u[k-1] + a \cdot e[k] + b \cdot e[k-1]$$

$$a = K_p + K_i \cdot T_s / 2$$

$$b = -K_p + K_i \cdot T_s / 2$$

Worked numeric example (this project)

$$T_s = 1/f_s = 1/10000.0 = 0.0001000 \text{ s.}$$

d-axis coefficients:

$$a_d = K_{p_d} + K_{i_d} \cdot T_s / 2 = 0.942478 + 6.283185 \cdot 0.0001000 / 2 = 0.942791955342$$

$$b_d = -K_{p_d} + K_{i_d} \cdot T_s / 2 = -0.942478 + 6.283185 \cdot 0.0001000 / 2 = -0.942163636812$$

q-axis coefficients:

$$a_q = K_{p_q} + K_{i_q} \cdot T_s / 2 = 1.413717 + 6.283185 \cdot 0.0001000 / 2 = 1.414030853381$$

$$b_q = -K_{p_q} + K_{i_q} \cdot T_s / 2 = -1.413717 + 6.283185 \cdot 0.0001000 / 2 = -1.413402534850$$

Saturation & anti-windup

Voltage saturation model

The saturation block limits the commanded voltage to a physically achievable phase-peak voltage based on DC-link voltage, PWM method, and modulation headroom. For SVPWM, the wizard uses:

$$V_{\text{phase_peak, max}} = m_{\text{max}} \cdot V_{\text{dc}} / \sqrt{3}$$

$$\text{With } V_{\text{dc}} = 700.0 \text{ V and } m_{\text{max}} = 0.95: V_{\text{phase_peak, max}} = 0.95 \cdot 700.0 / \sqrt{3} = 383.938 \text{ V.}$$

Anti-windup options implemented

Two anti-windup methods are supported in the time-domain check:

- Clamp (integrator freeze): when the PI output saturates, and the error would further increase saturation, the integrator update is halted.
- Back-calculation: an additional feedback term drives the integrator to match the applied (saturated) output, using a back-calculation gain k_{aw} .

Clamp anti-windup (mathematics)

Let $u = u_p + x_i$, where $u_p = K_p \cdot e$ and x_i is the integrator state. If saturation applies $u_{\text{sat}} = \text{sat}(u)$, then clamp AW updates x_i only if it helps bring u back inside the limit. Practically: freeze x_i when $|u| \geq u_{\text{max}}$ and $\text{sign}(e)$ matches $\text{sign}(u)$ (i.e., error pushes further into saturation).

Back-calculation anti-windup (mathematics)

Back-calculation modifies the integrator update by feeding back the difference between saturated and unsaturated outputs:

$$x_i[k+1] = x_i[k] + K_i \cdot T_s \cdot e[k] + k_{aw} \cdot (u_{sat}[k] - u[k])$$

If k_{aw} is too low, windup persists; if too high, the integrator can become noisy or inject non-physical dynamics. Clamp is simpler and often sufficient for current loops, provided the saturation model and sign logic are correct.

Robustness & Monte Carlo Analysis — Purpose, Method, and Interpretation

(A) Why is Monte Carlo used in this tool

Automotive traction drives experience large parameter and implementation variability. A single nominal PM/GM computed at one “best guess” plant is not enough to guarantee stability across temperature, current, and production spread. Monte Carlo analysis is used to answer: “If R_s , L_d/L_q , and delay shift within realistic bounds, what is the distribution of stability and sensitivity, and what is the worst case?”

Practical uncertainty drivers captured by this wizard:

- R_s drift with temperature (copper temperature coefficient; hot stator increases R_s).
- L_d/L_q variation with saturation and operating point (current-dependent inductances).
- Effective delay uncertainty (sampling phase, measurement filtering, compute load, PWM timing, actuator dynamics).
- Discretization and implementation details (incremental PI, quantization, filtering) that slightly shift effective loop dynamics.

Why nominal PM/GM is not enough: a loop with 60° PM nominal can still produce unacceptable ripple if the 5th-percentile PM falls near 45° or if sensitivity peaks (M_s) grow in tail cases. In production, it is the tails (worst-case) that determine field robustness.

(B) What exactly is randomized (with definitions)

From the robustness sweep UI (and code), the following parameters are randomized as independent multipliers applied to the nominal values:

Parameter	Symbol	Value	Units / Notes
<i>Stator resistance multiplier</i>	m_R	0.70 ... 1.50	Uniform random; R_s , sim = $m_R \cdot R_s$
<i>d-axis inductance multiplier</i>	m_{Ld}	0.80 ... 1.20	Uniform random; L_d , sim = $m_{Ld} \cdot L_d$
<i>q-axis inductance multiplier</i>	m_{Lq}	0.80 ... 1.20	Uniform random; L_q , sim = $m_{Lq} \cdot L_q$
<i>Delay multiplier</i>	m_{Td}	0.80 ... 1.20	Uniform random; T_d , sim = $m_{Td} \cdot T_d$

Uncertainty model: Uniform sampling within the specified min/max bounds. Sampling is independent (no correlations modeled).

TBD (if you want correlation): to model coupled effects (e.g., hot stator increases R_s and may reduce effective PWM performance), you would need a correlation model or measured joint distributions.

(C) How the sampling is performed

Number of Monte Carlo samples: $N = 400$. Random seed: 12345 (enables repeatable results).

For each sample $i = 1 \dots N$, the wizard performs the following sequence (per axis):

- 1) Draw multipliers (m_R , m_{Ld} , or m_{Lq} , m_{Td}).
- 2) Build sample plant $G_i(s) = 1/(L_i \cdot R_i)$.
- 3) Apply the delay model with T_d .
- 4) Build open-loop $L_i(s) = C(s) \cdot G_i(s) \cdot D_i(s)$.
- 5) Compute stability margins (PM, GM) and sensitivity peak metrics (M_s , M_t).
- 6) Store metrics; after N samples, plot histograms and compute worst-case / percentiles.

(D) How to interpret the Monte Carlo plots

Phase margin histogram (PM): x-axis in degrees. Higher is more stable/robust to unmodeled dynamics. “Tails” toward low PM indicate risk of oscillatory current, ripple, or instability in some parameter combinations.

Sensitivity peak histogram (M_s): x-axis shown in dB ($20 \cdot \log_{10}(\max| |)$). Lower is better. High M_s means the closed loop is “peaky”: disturbances/noise near the peak frequency are amplified, often observed as increased current ripple or ringing.

Suggested engineering guidelines (NOT hard-coded tool requirements; use your internal standards):

- PM: aim median $\geq 55\text{--}65^\circ$, and 5th-percentile $\geq 45\text{--}50^\circ$ for production.
- GM: $\geq 2\times$ (≈ 6 dB) is a common minimum.
- M_s (dB): keep worst-case < 6 dB ($\approx 2\times$) for current loops unless a faster response is explicitly required and verified.

Percentile use: the 5th percentile of PM (or 95th percentile of M_s) is often a better “robustness indicator” than the mean. Always review worst-case samples when you see long tails.

(E) Actionable design decisions from Monte Carlo

If Monte Carlo shows weak tails (low PM or high M_s), the corrective actions are typically:

- Reduce target crossover f_c (most effective): lower ω reduces delay phase lag and reduces sensitivity peaks.
- Increase PM target (if tool supports): forces a lower f_c , max, or more conservative selection.
- Revisit delay budget: measure effective delays (ADC, filtering, compute, PWM) rather than assuming optimistic values.
- Add filtering (carefully): improves noise but increases delay—so it can help or hurt; re-run Monte Carlo after changes.

- Update uncertainty ranges with data: use hot/cold R_s measurements, L maps vs current, and ECU timing logs.

Plot-by-Plot Interpretation Guide (Wizard Outputs)

This section explains how to interpret each plot provided by the wizard. The figures included below are the screenshots you provided.

Frequency-domain plots

Plant magnitude: dq voltage $\rightarrow$ dq current

What is plotted: $|G_d(j\omega)|$ and $|G_q(j\omega)|$ in dB, where $G(j\omega) = I(j\omega)/V(j\omega)$ for the RL model.

Why it matters: this is the base plant; it indicates where inductance dominates (-20 dB/dec slope) and how the d/q axes differ (L_d vs L_q).

What to look for: a lower L (typically the d-axis in this setup) gives a higher bandwidth (a higher corner frequency). A large separation between axes means you should not blindly reuse gains across axes.

Problem signs: unexpected resonances or non-first-order behavior would mean the RL model is insufficient (TBD if additional dynamics exist).

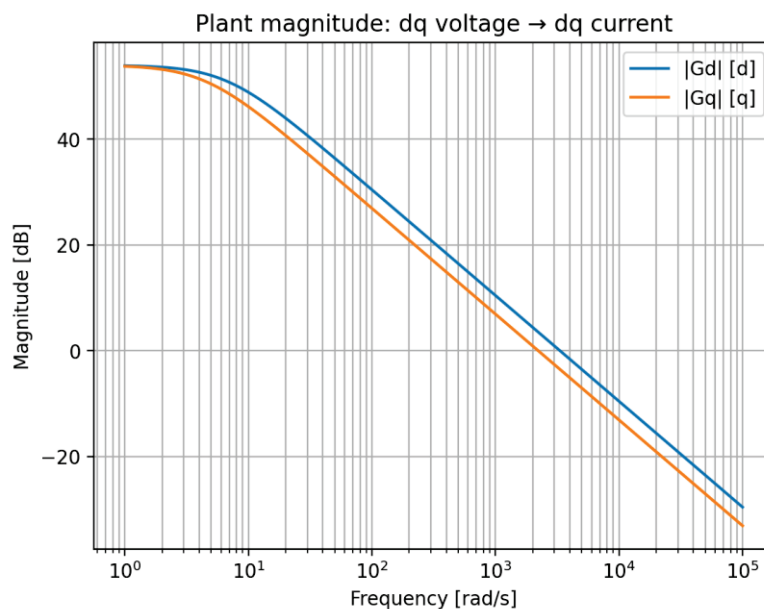


Figure 1 — Plant magnitude: dq voltage $\rightarrow$ dq current

Open-loop magnitude $|L(j\omega)|$

What is plotted: $|L(j\omega)|$ in dB after applying PI and plant. The 0 dB crossing is the crossover frequency f_c (approx).

Why it matters: It directly relates to bandwidth and disturbance rejection.

What to look for: a single 0 dB crossing with decent slope; if delay is included, very high f_c can create multiple crossings/peaking.

Problem signs: a crossover too close to sampling/PWM frequencies or with a shallow slope can yield poor margins.

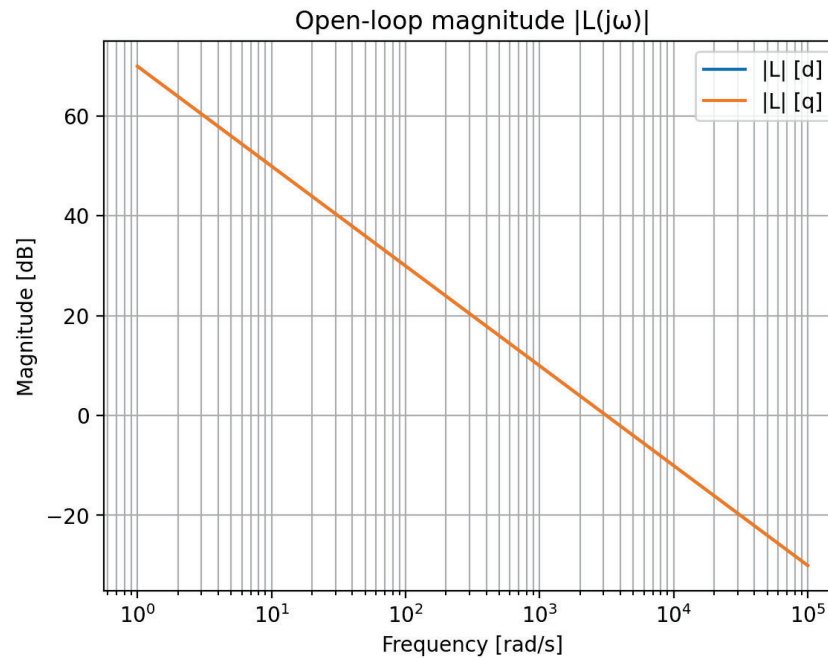


Figure 2 — Open-loop magnitude $|L(j\omega)|$

Sensitivity magnitude $|S(j\omega)|$

What is plotted: $|S(j\omega)| = |1/(1+L(j\omega))|$ in dB. Low-frequency values are strongly negative (good tracking/disturbance rejection).

Why it matters: the peak of $|S|$ is M_s . M_s bounds disturbance amplification and is strongly linked to current ripple/ringing in practice.

What to look for: M_s not too high (typically < 6 dB). Peak location indicates the “ringing” frequency.

Problem signs: M_s peaking > 6 –8 dB or broad peaking indicates fragile loop and likely ripple under real delays/noise.

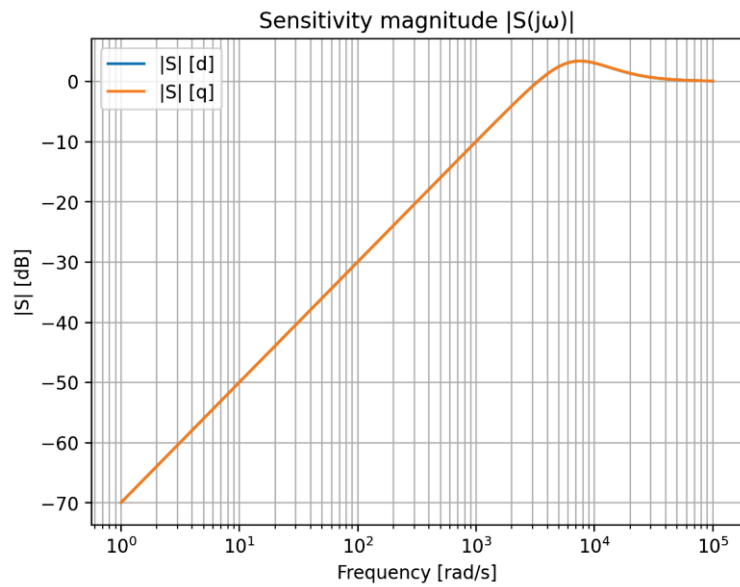


Figure 3 — Sensitivity magnitude $|S(j\omega)|$

Robustness sweep distributions

Phase margin distribution

What is plotted: histogram of PM across Monte Carlo samples for d- and q-axes.

How to use it: evaluate worst-case (left tail) and median. If the left tail approaches your minimum acceptable PM, reduce f_c or improve the delay budget.

Tool gating shown in UI: $PM_{min} = 50.0^\circ$ (used for PASS/FAIL messaging).

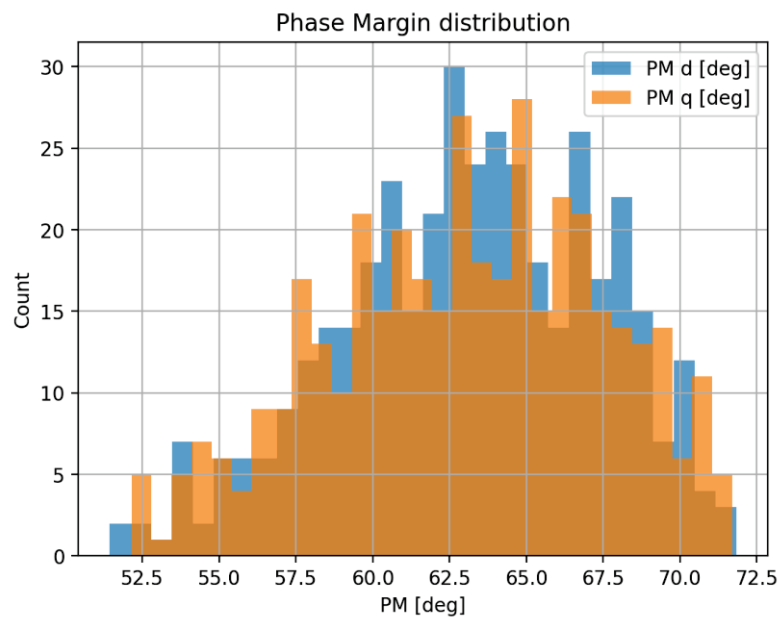


Figure 4 — Phase margin distribution (Monte Carlo)

Sensitivity peak Ms distribution

What is plotted: histogram of Ms across Monte Carlo samples. Ms is reported in dB ($20 \cdot \log_{10}(\max|S|)$).
How to use it: focus on the right tail (worst-case). High Ms means strong amplification near the peak frequency → ripple risk.

Tool gating shown in UI: $Ms_{\max} = 6.00$ dB.

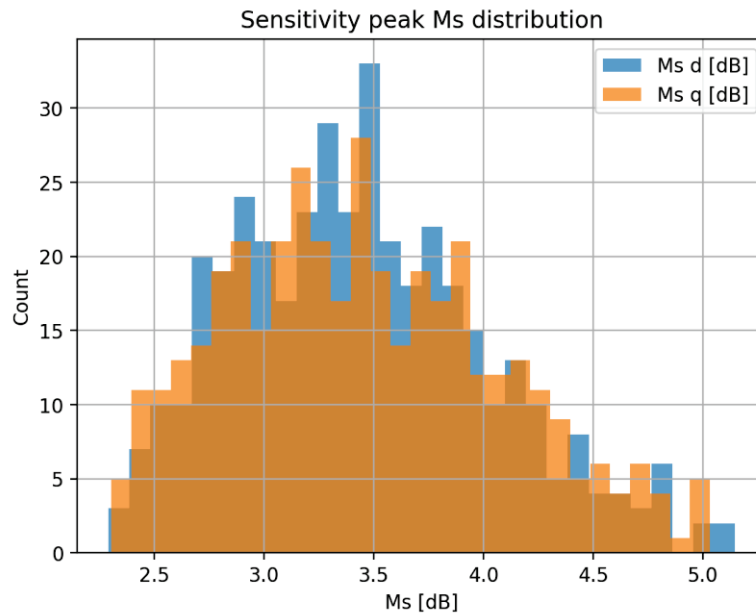


Figure 5 — Sensitivity peak Ms distribution (Monte Carlo)

Time-domain saturation / anti-windup plots

Current step response: no AW vs clamp

What is plotted: commanded current reference and simulated current response for a step, comparing anti-windup modes.

Why it matters: With voltage saturation, a PI without AW can wind up and overshoot or recover slowly. Clamp AW reduces windup and improves recovery.

What to look for: overshoot, settling time, and whether the response exhibits a “tail” due to windup.

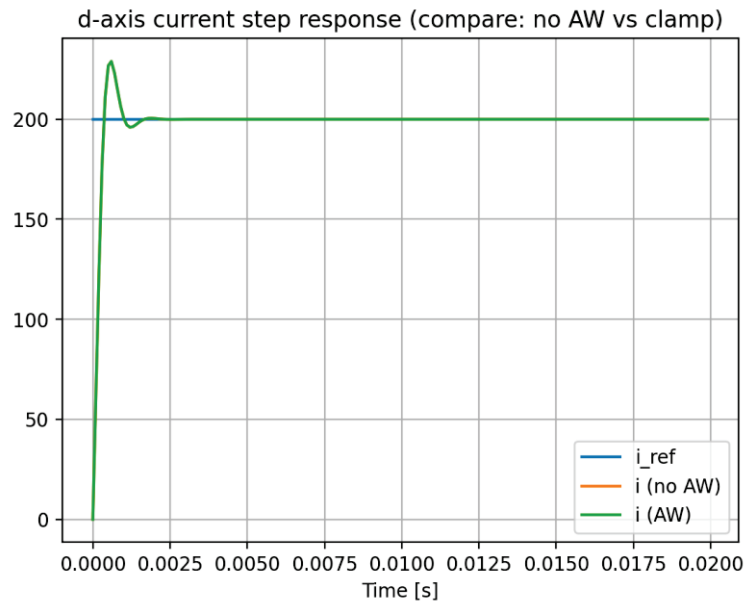


Figure 6 — d-axis step response: no anti-windup vs clamp

Voltage command vs saturation vs applied (no AW)

What is plotted: v_{cmd} (raw PI output), v_{sat} (clipped), and $v_{applied}$ (used by plant) under no anti-windup.

Why it matters: when $v_{cmd} \gg v_{sat}$, the integrator continues accumulating error, storing energy that later discharges as overshoot/ripple.

Problem sign: large, sustained difference between v_{cmd} and v_{sat} during saturation.

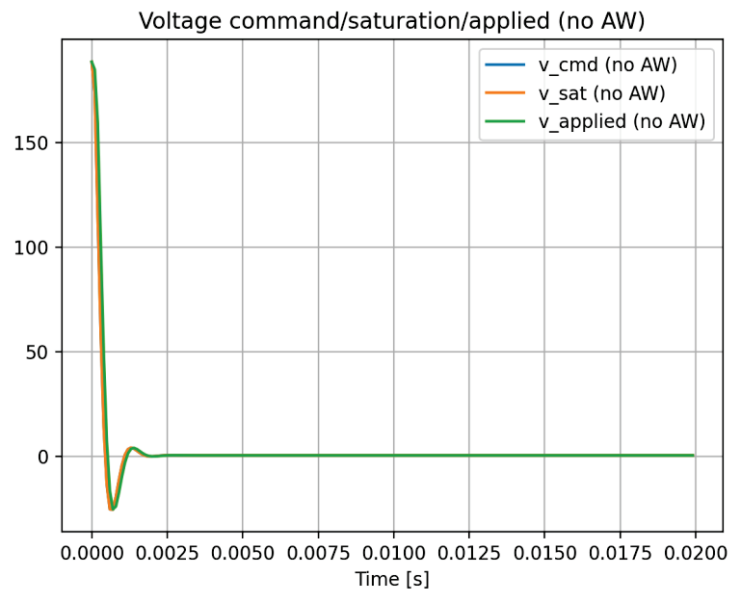


Figure 7 — Voltage command/saturation/applied (no AW)

Voltage command vs saturation vs applied (clamp)

Same signals as previous, but with clamp AW.

What to look for: v_{cmd} should stay closer to v_{sat} during saturation (integrator freeze prevents runaway). This typically reduces overshoot and tail.

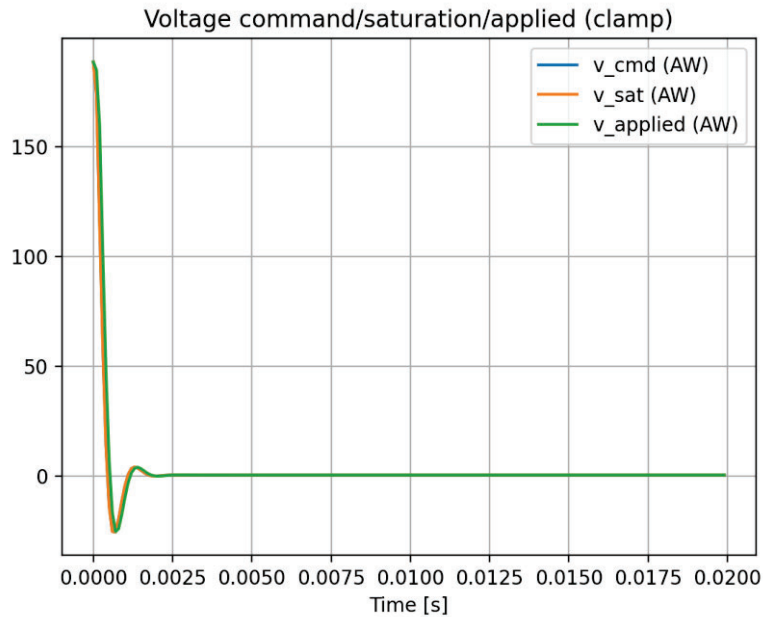


Figure 8 — Voltage command/saturation/applied (clamp AW)

Integrator state comparison

What is plotted: integrator state x_i (or x_l) with and without anti-windup.

Interpretation: without AW, x_i ramps during saturation and can take a long time to unwind. With a clamp, x_i is bounded/frozen, improving recovery.

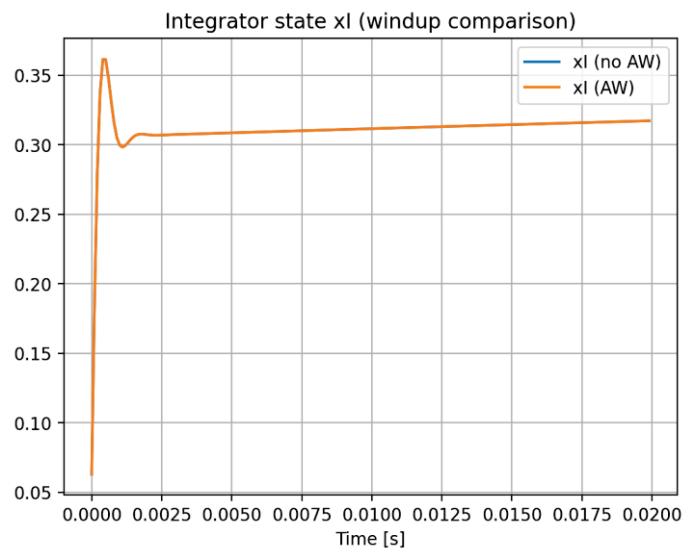


Figure 9 — Integrator state (windup comparison)

Wizard UI screenshots — what each input means

The following UI screenshots capture the key wizard pages and the chosen inputs for this project. Each page directly affects calculations.

The screenshot shows the 'Project Setup' page of the 'Current Controller Design Wizard'. The left sidebar contains a navigation menu with options: 'Intro', 'Project Setup' (selected), 'Plant / Model', 'Tuning (Preview)', 'Stability & Margins (Nominal)', 'Robustness Sweep (Monte Carlo)', 'Saturation & Anti-Windup', and 'Check'. The main content area is titled 'Current Controller Design Wizard' and includes a subtitle 'Guided workflow for dq current control design (robust, insensitive, self-tuning)' and 'Author: Masoud Sabahi'. The '1) Project Setup' section contains several input fields: 'Project ID' (PMSM_CurrentCtrl_V1), 'Sampling frequency [Hz]' (10000.00), 'Current unit' (A_peak), 'Name' (PMSM), 'PMSM method' (SVPWM), and 'Voltage unit' (V_phase). The 'Delays (implementation reality)' section includes 'PMSM delay [s]' (0.0000000), 'Computation delay [s]' (0.0000000), 'Measurement delay [s]' (0.0000000), and 'Total delay [s]' (1.500000e-04). The 'Run validation checks' section has a 'Run Checks' button and a green status bar indicating 'PAS Configuration is consistent'. A blue tip box at the bottom states: 'Tip: Keep in mind: (PMSM = compute + filtering). Too optimistic delay assumptions are a top cause of instability/regged current limits.'

Figure 10 — Wizard UI: Project Setup (f_s , PWM method, units, delay budget)

The screenshot shows the 'Plant / Model' page of the 'Current Controller Design Wizard'. The left sidebar is the same as in Figure 10, with 'Plant / Model' selected. The main content area is titled 'Current Controller Design Wizard' and includes the same subtitle and author information. The '2) Plant / Model (dq RL)' section contains input fields for 'Rs [ohm]' (0.00200000), 'Ld [H]' (0.00000000), 'Lq [H]' (0.00400000), and 'J [kg*m^2]' (0.00000000). The 'Validation + quick plant insights' section shows 'Tm, L+ [s]' (1.500000e-01) and 'Tm, L+ [s]' (2.250000e-01).

Figure 11 — Wizard UI: Plant / Model (R_s , L_d , L_q ; time constants)

The screenshot shows the 'Tuning (Preview)' page of the 'Current Controller Design Wizard'. The left sidebar is the same as in Figure 10, with 'Tuning (Preview)' selected. The main content area is titled 'Current Controller Design Wizard' and includes the same subtitle and author information. The '5) Tuning (Preview) — Delay-aware bandwidth PI' section contains input fields for 'Target crossover [Hz]' (500.00), 'PI target bandwidth [Hz]' (50.00), and 'Desired closed-loop time [s]' (555.6). The 'Proposed nominal continuous-time gains (bandwidth PI)' section shows two code blocks. The first block contains:

```
{
  "kp_p1": 0.341477788078326,
  "ki_p1": 0.28322504717938
}
```

. The second block contains:

```
{
  "kp_p2": 1.41270089410441,
  "ki_p2": 0.28322504717938
}
```

Figure 12 — Wizard UI: Tuning (Preview) selected crossover and proposed gains

Units & scaling sanity checks (common pitfalls)

The wizard intentionally forces explicit unit choices because the dq current control is sensitive to RMS/peak and voltage definitions. A few high-impact checks:

- Current: A_{peak} vs A_{rms} . If the controller is tuned in A_{peak} but logs or ECU signals are A_{rms} , gains, and saturation checks will be inconsistent.
- Voltage: $V_{\text{phase_peak}}$ vs $V_{\text{ll_rms}}$. The saturation model in this wizard assumes $V_{\text{phase_peak_max}}$; converting from DC link must match your modulation method.
- Delay: use seconds everywhere. “150 μs ” must be entered as 150e-6 s. Delay mistakes are a top cause of overly aggressive bandwidth and ripple.
- Sampling: ensure f_s matches the actual current-control execution rate (after any decimation).

Engineering notes and limitations

The RL tuning model is intentionally simple and robust as a first-order starting point. In real IPMSM traction systems, the following effects can materially change behavior and must be validated:

- Cross-coupling and back-EMF: dq axes are coupled by $\omega_e \cdot L$ terms and $\omega_e \cdot \psi_f$. Decoupling/feedforward quality affects “effective plant” and bandwidth.
- Saliency and saturation: L_d and L_q vary with current and temperature; the Monte Carlo sweep approximates this via multipliers. Use measured L maps to tighten ranges.
- Measurement chain: ADC + filtering adds delay and noise shaping; if additional low-pass filters exist, include them in the delay budget or explicit model.
- PWM non-idealities: deadtime, voltage drops, and limited resolution distort v_{applied} ; this often shows as steady-state error or ripple at high modulation.
- Quantization and scheduling jitter: can reduce effective PM; watch worst-case samples.

Traceability (math/metrics → code)

The table below links key equations/metrics in this report to the implementation in CurrentCtrlWizard.py.

Equation / Function Metric		Key inputs	Outputs
Voltage limit (SVPWM)	$V_{\text{phase_peak_max}}()$	Vdc, m_{max} , pwm_method	$V_{\text{phase_peak_max}}$
Bandwidth PI gains	tune_bandwidth_pi ()	R, L, f_c	Kp, Ki
Delay-limited f_c , max	f_c_delay_limited ()	T_d , PM_target	f_c , ma
Discrete PI (Tustin incremental)	discrete_pi_tustin_incremental ()	Kp, Ki, T_s	a, b coefficients
Open-loop construction	build_open_loop ()	Plant + PI + delay	L(s)
Margins (PM/GM)	compute_margins ()	L(j ω)	PM, GM
Sensitivity peaks (Ms/Mt)	compute_sensitivity_peaks()	S(j ω), T(j ω)	Ms, Mt
Monte Carlo sweep	page_robustness_monte_carlo ()	Uncertainty bounds, N, seed	Histograms + pass-rate
Saturation/AW time sim	simulate_sat_aw_step ()	Kp, Ki, Vmax, T_d , AW mode	Step response + signals

Calibration & deployment notes

Recommended practical workflow (dyno/vehicle):

- 1) Validate delay budget: measure current-loop timing, ADC/filter delay, PWM update timing. Update T_d and re-tune if needed.
- 2) Validate L maps: identify L_d/L_q vs current (and temperature if possible). Update Monte Carlo ranges to match measured extremes.
- 3) Start with conservative f_c (e.g., 0.7–0.9 of delay-limited max). Verify ripple and step response across speed and modulation.
- 4) Check saturation behavior: at high speed/torque, the voltage margin shrinks. Ensure AW prevents large

overshoots and that current limits/protections are respected.

5) Iterate: adjust f_c and filters based on measured ripple and stability indicators.

Common failure modes and troubleshooting

- Excess ripple after increasing gains: often indicates reduced PM due to unmodeled delay/filtering, or excitation of resonances; reduce f_c or fix delay budget.
- Slow recovery after saturation: AW missing/mis-signed; verify clamp/back-calc logic and voltage limit correctness ($V_{\text{phase_peak}}$ vs V_{ll}).
- d/q mismatch: using the same gains despite different L_d/L_q ; tune per axis as the wizard does.
- Monte Carlo PASS but real system fails: uncertainty ranges too optimistic or missing dynamics (filters, decoupling errors). Expand ranges and include additional delays.

What changed / what to review (v3 checklist)

- Verify that R_s , L_d , L_q , f_s , V_{dc} , m_{max} , and delay components match your real project configuration.
- Confirm the delay-limited crossover formula matches your internal margin targets and delay modeling conventions.
- Review Monte Carlo uncertainty ranges; replace multipliers with data-driven ranges if available.
- Check unit consistency (A_{peak} vs A_{rms} ; $V_{\text{phase_peak}}$ vs V_{ll}).
- If your ECU includes additional current filters or decimation, add their delay and re-run the stability/robustness evaluation.